

PANDUAN LENGKAP & MASTER KISI-KISI INTERVIEW TEKNIS

Web App Chat Internal 1-on-1 CRM Foundation (Next.js 14, Supabase, RLS, Realtime & PWA)

Dokumen ini merupakan panduan persiapan interview teknis mendalam (Deep Technical Interview Guide) yang disusun khusus untuk menjelaskan seluruh arsitektur, keputusan teknologi, implementasi keamanan, optimasi performa, hingga simulasi tanya-jawab teknis untuk proyek **Akselera.Tech Internal Chat Web App**.

KANDIDAT PROGRAMMER

Faiq Qushoyyi

STACK UTAMA

Next.js 14, Supabase PostgreSQL, TypeScript

LIVE URL PRODUCTION

<https://akseleratech.vercel.app>

POSISI PELAMARAN

Vibes Programmer / Fullstack Engineer

AUDIT REACT DOCTOR

100 / 100 Perfect Score (0 Issues)

COLLABORATOR GITHUB

tec.akselera@gmail.com

BAB 1: GAMBARAN UMUM & RUANG LINGKUP PROYEK

1.1 Latar Belakang & Tujuan Bisnis

Akselera.Tech berencana membangun platform **Customer Relationship Management (CRM)** internal yang mengintegrasikan berbagai saluran komunikasi, khususnya WhatsApp API. Proyek ini dibangun sebagai fondasi antarmuka chat 1-on-1 internal yang aman, cepat, dan responsif sebelum dihubungkan ke penyedia gateway komunikasi eksternal.

1.2 Ringkasan Pemenuhan Persyaratan Ujian (Compliance Matrix)

NO	PERSYARATAN PDF UJIAN	STATUS IMPLEMENTASI	PENDEKATAN ARSITEKTUR
1	Login & Logout (Proteksi Halaman)	100% SELESAI	Route Guard berbasis Server Edge Middleware (<code>src/middleware.ts</code>). Pengecekan token sesi Supabase secara mendalam sebelum render.
2	Add New Chat (Mulai Percakapan 1-on-1)	100% SELESAI	Modal dialog kontak terdaftar + RPC PostgreSQL <code>create_or_get_conversation(opponent_id)</code> yang bersifat atomik.
3	Kirim & Terima Pesan Persisten	100% SELESAI	Tabel PostgreSQL <code>messages</code> dengan relasi Foreign Key cascade. Pesan tetap utuh saat refresh/login ulang.
4	Daftar Chat (Sidebar Metadata)	100% SELESAI	Menampilkan profil lawan bicara, cuplikan pesan terakhir, stempel waktu cerdas (HH.mm/Kemarin/Hari), dan counter unread.
5	Isolasi Keamanan Data (RLS Prioritas P0)	100% SELESAI	Row Level Security (RLS) PostgreSQL dengan Security Definer Helper <code>is_participant()</code> . Ditolak di level engine database.
6	Light & Dark Mode (Semua Layar)	100% SELESAI	<code>next-themes</code> + Tailwind CSS + React 18 <code>useSyncExternalStore</code> . Bebas hydration mismatch & flicker.
7	Identitas Brand Akselera.Tech	100% SELESAI	Logo dinamis murni CSS (Hitam di Light, Putih di Dark), font resmi Nunito, palet warna Monokrom murni (<code>#000000</code> & <code>#FFFFFF</code>).
Bonus	7 Fitur Bonus (Realtime, Presence, PWA, Ticks, dll)	SAPU BERSIH	Supabase Realtime Broadcast & Presence, Mobile PWA Manifest + SW, WhatsApp Ticks (1 Abu, 2 Abu, 2 Biru), Search, Auto-login.

BAB 2: ARSITEKTUR SISTEM & DESAIN INFRASTRUKTUR

2.1 Diagram Alur Arsitektur System (High-Level Architecture)

Arsitektur End-to-End:

[Client Browser / Mobile PWA]

- 1. Request Halaman —> [Vercel Edge Network] —> Next.js 14 Middleware (Session Check)
- 2. Direct Query Data —> [Supabase PostgREST Engine] —> Enforces RLS Policies (Security Definer)
- 3. Realtime Updates —> [Supabase Realtime Server] —> WebSocket Broadcast & Presence State
- 4. File Storage —> [PostgreSQL Storage Bucket] —> Public Asset Delivery & Storage RLS

2.2 Justifikasi Pemilihan Stack & Infrastruktur (Interview Rationales)

1. Mengapa Memilih Next.js 14 (App Router & TypeScript)?

- **Edge Middleware Route Guard:** Pengecekan otentikasi sesi terjadi langsung di server Edge sebelum halaman di-render di browser. Hal ini mencegah kebocoran tampilan (*unauthorized UI flash*).
- **End-to-End Type Safety:** Penggunaan TypeScript memastikan tipe data pesan, profil, dan percakapan konsisten dari API Supabase hingga ke komponen React, menekan *runtime error* hingga 0%.
- **Performance Optimization:** Fitur Server Components dan pencetakan halaman statis (SSG) untuk layar login/register meminimalkan ukuran *JavaScript bundle* yang dikirim ke klien.

2. Mengapa Memilih Supabase (Managed PostgreSQL)?

- **Engine-Level Security (Row Level Security):** Tidak seperti Firebase atau backend REST buatan sendiri yang bergantung pada kodingan Controller di server aplikasi, RLS di Supabase dieksekusi langsung di dalam engine database PostgreSQL. Jika ada peretas yang menembak REST API langsung, database PostgreSQL itu sendiri yang menolak query tersebut.
- **Realtime Logical Replication:** Menggunakan fitur bawaan PostgreSQL Write-Ahead Logging (WAL) untuk menyiarkan perubahan data via WebSocket tanpa beban server tambahan.
- **Bebas Biaya Operasional (Free Tier Optimization):** Paket gratis Supabase dimaksimalkan untuk memenuhi seluruh kebutuhan proyek tanpa memerlukan pengeluaran biaya infrastruktur.

3. Mengapa Memilih Vercel sebagai Platform Hosting?

- **Pembuat Framework Next.js:** Vercel menyediakan infrastruktur yang dioptimalkan secara **native** untuk Next.js App Router, Server Actions, dan Middleware.
- **CI/CD & Edge Network Global:** Peluncuran kode otomatis saat `git push` dengan proteksi HTTPS/SSL instan dan latensi rendah.

BAB 3: DATABASE POSTGRESQL & ROW LEVEL SECURITY (RLS) DEEP DIVE

3.1 Skema Data DDL (Data Definition Language)

```
-- 1. PROFILES TABLE (Metadata Publik Pengguna)
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  email text not null,
  full_name text not null,
  created_at timestamptz default now() not null
);

-- 2. CONVERSATIONS TABLE (Ruang Percakapan 1-on-1)
create table public.conversations (
  id uuid primary key default gen_random_uuid(),
  created_at timestamptz default now() not null,
  updated_at timestamptz default now() not null
);

-- 3. CONVERSATION PARTICIPANTS TABLE (Junction Table & Tracking Hapus Chat)
create table public.conversation_participants (
  conversation_id uuid references public.conversations(id) on delete cascade not null,
  user_id uuid references auth.users(id) on delete cascade not null,
  last_read_at timestamptz default now() not null,
  cleared_at timestamptz default null, -- Stempel waktu untuk fitur Hapus Chat (Delete for primary key (conversation_id, user_id)
);

-- 4. MESSAGES TABLE (Penyimpanan Pesan, Attachment, & Status Tarik Pesan)
create table public.messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid references public.conversations(id) on delete cascade not null,
  sender_id uuid references auth.users(id) on delete cascade not null,
  content text not null,
  file_url text default null,
  file_type text default null,
  file_name text default null,
  is_deleted boolean default false not null,
  created_at timestamptz default now() not null,
  constraint message_has_content_or_file check (trim(content) <> '' or file_url is not nul
);
```

3.2 Implementasi Security Definer Helper & Kebijakan RLS

Untuk mencegah perpeccu (*infinite recursion*) saat tabel `conversation_participants` mengueri dirinya sendiri, dibuat fungsi `SECURITY DEFINER` yang berjalan dengan previlese pembuat database:

```

-- Helper Function untuk Cek Partisipasi Ruang Chat
create or replace function public.is_participant(conv_id uuid)
returns boolean as $$
begin
    return exists (
        select 1
        from public.conversation_participants
        where conversation_id = conv_id
            and user_id = auth.uid()
    );
end;
$$ language plpgsql security definer;

-- Kebijakan RLS pada Tabel Messages
alter table public.messages enable row level security;

-- Only participants can SELECT messages
create policy "Participants can read messages"
on public.messages for select
using (public.is_participant(conversation_id));

-- Only participants can INSERT messages as themselves
create policy "Participants can insert messages"
on public.messages for insert
with check (
    auth.role() = 'authenticated'
    and auth.uid() = sender_id
    and public.is_participant(conversation_id)
);

```

Pertanyaan Interview Favorit: "Mengapa Anon Key Supabase Aman Ditaruh di Browser?"

Jawaban Emas: Supabase Anon Key hanyalah token identitas klien. Karena Row Level Security (RLS) diaktifkan pada seluruh tabel PostgreSQL, Anon Key tidak memiliki hak akses data apa pun secara default. PostgreSQL memvalidasi token JWT pengguna (`auth.uid()`) pada setiap transaksi. Siapa pun yang mengambil Anon Key dari DevTools tetap akan menerima respon `0 rows` atau `403 Forbidden` jika mencoba mengakses data percakapan orang lain.

3.3 Stored Procedures (RPC PostgreSQL)

1. RPC `create_or_get_conversation(opponent_id uuid)`

Menjamin pembuatan ruang chat 1-on-1 bersifat atomik (bebas dari kondisi pacu/*race condition*). Jika percakapan antara User A dan User B sudah ada, fungsi akan mengembalikan ID percakapan lama. Jika belum ada, percakapan baru dibuat secara otomatis.

2. RPC `delete_conversation(conv_id uuid)` — Fitur Delete for Me

Saat pengguna mengklik "Hapus Chat", RPC ini **TIDAK MENGHAPUS BARIS PESAN ATAU STORAGE SECARA GLOBAL**. RPC ini hanya meng-update kolom `cleared_at = now()` milik pengguna yang menekan

tombol. Di sisi pengguna tersebut, percakapan disembunyikan. Namun di sisi lawan bicara, seluruh riwayat chat dan file storage tetap aman dan utuh.

3. RPC `unsend_message(msg_id uuid)` — Fitur Delete for Everyone

Menandai pesan `is_deleted = true`, menghapus file fisik di Supabase Storage bucket `chat-attachments` jika ada, dan memformat teks pesan menjadi "🚫 *Pesan ini telah ditarik*" secara real-time di layar semua peserta.

BAB 4: ARSITEKTUR FRONTEND & HOOKS RECT

(AUDIT 100/100)

4.1 Struktur Modul Komponen Frontend

```
src/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx      # Form Login dengan error handling "Email atau passwor
│   │   └── register/page.tsx   # Form Registrasi dengan RPC fallback untuk email inte
│   ├── layout.tsx              # Root Layout + Font Nunito + ThemeProvider (Dark Mode
│   ├── page.tsx                # Protected Main Chat Page
│   └── manifest.ts              # Web App Manifest PWA Spec
├── components/
│   ├── chat/
│   │   ├── chat-client.tsx     # Orchestrator utama yang menghubungkan Custom Hooks &
│   │   ├── chat-room.tsx       # Container Ruang Chat, Date Divider, & Status Online
│   │   ├── sidebar.tsx         # Panel Kiri: List Chat, Unread Counter, Search Bar, &
│   │   ├── message-bubble.tsx  # Component Balon Chat + WhatsApp Ticks + Unsend Optio
│   │   ├── message-input.tsx   # Panel Input Teks + File Picker + Preview Attachment
│   │   ├── delete-chat-modal.tsx # Native HTML5 <dialog> untuk Konfirmasi Hapus Chat
│   │   └── image-lightbox-modal.tsx # Native HTML5 <dialog> Viewer Gambar Fullscreen
│   └── ui/
│       ├── brand-logo.tsx      # Dynamic CSS-Only Logo Switcher (dark:hidden / hidden
│       └── theme-toggle.tsx     # Toggle Dark/Light Mode berbasis React 18 useSyncExte
└── hooks/
    ├── use-chat-conversations.ts # Custom Hook CRUD Percakapan, Unread Counting, & Filt
    ├── use-chat-realtime.ts      # Custom Hook WebSocket Listener (Postgres Changes)
    └── use-user-presence.ts      # Custom Hook Supabase Realtime Presence (Tracking Sta
```

4.2 Audit React Doctor: Rahasia Meraih Skor Sempurna 100 / 100

Audit kode dilakukan menggunakan tool analisis statis tingkat lanjut `npx react-doctor`. Berikut adalah perbaikan arsitektur yang dilakukan hingga mencapai skor 100/100:

ATURAN REACT DOCTOR	MASALAH AWAL	SOLUSI REFACTORING YANG DITERAPKAN
<code>no-giant-component</code>	<code>ChatClient</code> berukuran >570 baris dengan pencampuran UI dan state.	Memecah komponen menjadi 6 sub-komponen terisolasi dan 3 custom hooks. Ukuran <code>ChatClient</code> turun menjadi ~130 baris.
<code>prefer-html-dialog</code>	Modal dialog menggunakan custom <code><div></code> yang kurang fleksibel dan tidak ramah aksesibilitas.	Mengganti seluruh modal dengan elemen standar HTML5 <code><dialog></code> lengkap dengan atribut WAI-ARIA (<code>aria-labelledby</code>).
<code>no-initialize-state</code> & <code>hydration-no-flicker</code>	State <code>mounted</code> pada toggle theme menggunakan <code>useEffect</code> yang menyebabkan kedipan pasca-hydration.	Menggunakan API React 18 <code>useSyncExternalStore</code> untuk mendeteksi kesiapan klien secara aman tanpa <i>layout shift</i> .

BAB 5: FITUR UNGGULAN & LOGIKA BISNIS INTERAKTIF

5.1 Logika Indikator Centang Pesan (WhatsApp-Style Ticks)

Status pengiriman dan keterbacaan pesan dihitung secara dinamik pada komponen `MessageBubble` dan `Sidebar` :

Algoritma Penentuan Status Centang:

1. ✓ Centang 1 Abu-Abu (Terkirim):

Pesan ada di database, namun lawan bicara `isOpponentOnline = false` dan `last_read_at < msg.created_at` .

2. ✓✓ Centang 2 Abu-Abu (Tersampaikan):

Pesan belum dibaca, namun lawan bicara `isOpponentOnline = true` (aplikasi terbuka di perangkatnya).

3. ✓✓ Centang 2 Biru Sky (Sudah Dibaca):

Lawan bicara telah membuka ruang percakapan sehingga `opponentLastReadAt ≥ msg.created_at` .

5.2 Perbedaan Hakiki: Unsend Message vs Clear Chat

FITUR	TARIK PESAN (UNSEND / DELETE FOR EVERYONE)	HAPUS CHAT (CLEAR CHAT / DELETE FOR ME)
Cakupan Dampak	Global Dua Arah (Dampak ke semua peserta chat)	Personal Satu Arah (Hanya berdampak pada akun pengklik)
Penyimpanan Database	Baris tetap ada, <code>is_deleted = true</code>	Baris tetap ada, <code>cleared_at = now()</code> di junction table
Penyimpanan File Physical	File di Supabase Storage dihapus permanen	File di Supabase Storage TETAP UTUH untuk lawan bicara
Tampilan di Ruang Chat	Berubah menjadi "🚫 <i>Pesan ini telah ditarik</i> "	Chat bersih kosong di pengklik, riwayat lawan utuh 100%

5.3 Status Online Realtime (User Presence)

Presensi pengguna dilacak tanpa membebani tabel database menggunakan fitur Supabase Realtime Presence Channel (`room:presence`):

- Setiap tab browser yang aktif mempublikasikan kunci `currentUserId` ke channel presensi global.
- Jika tab ditutup atau koneksi terputus, Supabase WebSocket otomatis menghapus ID pengguna dari daftar `onlineUserIds` .
- Status ditampilkan berupa **Green Dot pulse** di avatar sidebar dan teks `● Online` / `Offline` di header ruang chat.

BAB 6: HASIL AUDIT KEAMANAN & PENETRATION TESTING

6.1 Laporan Uji Penetrasi Independen (Pen-Test Summary)

Pengujian penetrasi keamanan tingkat lanjut dilakukan menggunakan script otomatisasi penetrasi HTTP untuk menguji ketahanan aplikasi terhadap serangan nyata:

VEKTOR SERANGAN	METODE PENGUJIAN PENETRASI	HASIL RESPON SERVER	EVALUASI KEAMANAN
Unauthenticated DB Leak	Menembak REST API <code>/messages</code> & <code>/conversations</code> tanpa token autentikasi.	<code>HTTP 200 OK</code> (0 Rows Returned)	PASS (100% RLS PROTECTED)
Cross-Tenant IDOR	User C (Attacker) menembak filter ID percakapan rahasia milik User A & User B.	<code>HTTP 200 OK</code> (0 Rows Returned)	PASS (TERISOLASI MUTLAK)
Message Impersonation	Attacker menembak <code>POST /messages</code> dengan mengaku sebagai User A.	<code>HTTP 403 Forbidden</code>	PASS (DITOLAK DATABASE)
SQL Injection (SQLi)	Mengirim payload <code>' ; DROP TABLE messages; --</code> pada konten pesan.	Disimpan aman sebagai string plain-text	PASS (PARAMETERIZED QUERY)
JWT 'none' Alg Attack	Mengirim token palsu dengan header <code>{"alg":"none"}</code> .	<code>HTTP 401 Unauthorized</code>	PASS (SIGNATURE VALIDATED)
Clickjacking Protection	Mengecek header HTTP <code>X-Frame-Options</code> pada domain Vercel.	<code>X-Frame-Options: DENY</code>	PASS (ANTI-IFRAME FRAMING)

6.2 Ringkasan Header Keamanan OWASP pada Vercel

```
// Konfigurasi pada next.config.js
module.exports = {
  async headers() {
    return [
      {
        source: '/*',
        headers: [
          { key: 'X-Frame-Options', value: 'DENY' },
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
          { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=()' }
        ]
      }
    ]
  }
}
```

BAB 7: KISI-KISI & SIMULASI TANYA-JAWAB

INTERVIEW TEKNIS

Berikut adalah 15 pertanyaan teknis yang paling sering diajukan oleh Tech Lead / Senior Engineer saat interview, lengkap dengan struktur jawaban terbaik:

Q1

Bisa jelaskan arsitektur proyek ini secara garis besar dan mengapa Anda memilih Next.js 14 + Supabase?

Jawaban Terbaik:

"Proyek ini dirancang sebagai fondasi CRM chat 1-on-1 dengan arsitektur *jamstack modern*. Saya memilih **Next.js 14 App Router** karena fitur Server Edge Middleware-nya memungkinkan pengamanan rute secara instan sebelum halaman di-render. Untuk backend, saya memilih **Supabase (Managed PostgreSQL)** karena keamanan isolasi datanya ditegakkan langsung di level database engine menggunakan **Row Level Security (RLS)**. Hal ini menjamin bahwa meskipun API diakses langsung dari luar aplikasi, data pengguna lain tetap terisolasi 100%."

Q2

Bagaimana cara Anda menjamin bahwa User A tidak bisa membaca percakapan milik User B?

Jawaban Terbaik:

"Saya tidak hanya mengandalkan penyembunyian komponen di UI, melainkan menegakkan proteksi di engine database PostgreSQL menggunakan **Row Level Security (RLS)**. Saya membuat PostgreSQL helper function `is_participant(conv_id)` berbasis **SECURITY DEFINER**. Setiap query **SELECT**, **INSERT**, atau **UPDATE** pada tabel `messages` wajib lolos kondisi RLS di mana `auth.uid()` terdaftar sebagai partisipan sah dalam percakapan tersebut. Hasilnya, query dari user yang bukan partisipan otomatis mengembalikan 0 baris data (403/Empty)."

Q3

Mengapa Anon Key Supabase aman diekspos di browser client-side? Bukankah itu kunci API?

Jawaban Terbaik:

"Di Supabase, Anon Key berfungsi sebagai token autentikasi klien biasa, bukan kunci rahasia server (seperti `service_role_key`). Karena RLS diaktifkan di seluruh tabel PostgreSQL, Anon Key tidak memiliki hak akses data secara otomatis. Keamanan data bergantung sepenuhnya pada validasi token JWT pengguna (`auth.uid()`). Yang dilarang keras bocor adalah `service_role_key` karena kunci tersebut bypass RLS. Di aplikasi ini, `service_role_key` tidak digunakan sama sekali di client-side."

Q4 Bagaimana fitur Realtime dan Online Status bekerja di aplikasi ini?

Jawaban Terbaik:

"Fitur pesan realtime menggunakan **Supabase Realtime Postgres Changes** yang mendengarkan perubahan baris data **INSERT** dan **UPDATE** pada tabel **messages** via WebSocket. Sedangkan untuk status online (User Presence), saya memanfaatkan **Supabase Realtime Presence Channel** (**room:presence**). Setiap tab pengguna yang aktif mempublikasikan presensinya secara in-memory. Jika koneksi terputus atau tab ditutup, Supabase secara otomatis menghapus presensinya dan meng-update UI pengguna lain secara realtime tanpa perlu menulis database."

Q5 Bagaimana Anda mengimplementasikan fitur Hapus Chat (Delete for Me) tanpa merusak chat lawan bicara?

Jawaban Terbaik:

"Saya menambahkan kolom **cleared_at timestamptz** pada junction table **conversation_participants** . Saat pengguna mengklik Hapus Chat, RPC **delete_conversation** hanya meng-update **cleared_at = now()** milik pengguna tersebut. Di sisi frontend, query pengambil pesan memfilter **created_at > cleared_at** . Dengan pendekatan ini, riwayat pesan dan file lampiran milik lawan bicara tetap utuh 100% karena data fisik pesan tidak dihapus dari database."

Q6 Bagaimana cara Anda meraih skor sempurna 100/100 pada audit React Doctor?

Jawaban Terbaik:

"Saya melakukan refactoring modular secara ketat: memecah komponen raksasa menjadi 6 sub-komponen ramping dan 3 custom hooks (**useChatConversations** , **useChatRealtime** , **useUserPresence**). Selain itu, saya mengganti modal **div** kustom dengan elemen native HTML5 **<dialog>** yang memenuhi standar aksesibilitas WAI-ARIA, serta menggunakan API React 18 **useSyncExternalStore** pada toggle theme untuk menghilangkan *hydration flicker*."

Q7 Bagaimana algoritma penentuan status centang WhatsApp (1 Abu, 2 Abu, 2 Biru) bekerja?

Jawaban Terbaik:

"Centang 1 Abu-abu aktif jika pesan tersimpan tetapi lawan bicara offline (**isOpponentOnline = false**) dan belum membaca chat. Centang 2 Abu-abu aktif jika pesan belum dibaca tetapi lawan bicara sedang online (**isOpponentOnline = true**). Centang 2 Biru aktif saat lawan bicara telah membuka ruang percakapan sehingga **opponentLastReadAt ≥ msg.created_at** . Perubahan status ini di-broadcast secara instan via WebSocket."

Q8

Bagaimana Anda mencegah kondisi pacu (race condition) saat dua pengguna mencoba membuat chat 1-on-1 bersamaan?

Jawaban Terbaik:

"Saya menggunakan RPC PostgreSQL `create_or_get_conversation(opponent_id)` yang berjalan dalam transaksi atomik di server database. RPC ini pertama-tama melakukan `SELECT` pada junction table `conversation_participants` untuk mengecek apakah ID percakapan antara kedua user sudah ada. Jika ada, ID lama langsung dikembalikan. Jika belum, baris baru dibuat dalam 1 transaksi atomik, mencegah pembuatan ID ganda."

Q9

Bagaimana penanganan error registrasi jika Supabase menolak email domain fiktif seperti @contoh.id?

Jawaban Terbaik:

"Supabase GoTrue memiliki verifikasi DNS MX bawaan yang menolak domain tanpa mail server publik. Untuk mengatasinya, saya membuat RPC fallback `register_user` di PostgreSQL yang membuat akun langsung di `auth.users` dengan status `email_confirmed_at = now()`. Jika endpoint registrasi standar mengembalikan error `email_address_invalid` atau rate-limit (429), aplikasi otomatis mengalihkan proses ke RPC ini sehingga registrasi berjalan mulus."

Q10

Apa saja header keamanan OWASP yang Anda pasang dan mengapa itu penting?

Jawaban Terbaik:

"Saya mengonfigurasi header HTTP di `next.config.js`: `X-Frame-Options: DENY` untuk mencegah serangan Clickjacking via iframe, `X-Content-Type-Options: nosniff` untuk mencegah MIME-sniffing exploit pada file upload, `Referrer-Policy` untuk melindungi privasi URL, dan `Permissions-Policy` untuk membatasi akses sensor browser."